Grupa: ..... Numele şi prenumele (cu MAJUSCULE): .....

Arhitectura Sistemelor de Calcul: Test - Laboratorul 0x04, 10% din nota finală

Exerciţiul 1 (1.5p) *Scrieţi dimensiunea alocată în RAM, atât în biţi, cât şi în Bytes, pentru următoarele variabile declarate în secţiunea* `.data`*:*

Declarare în .data	dimensiune în biţi	dimensiune în Bytes
`str1: .ascii "Test ASC L0x04\n"`	120	15
`str2: .asciz "Test ASC L0x04\n"`	128	16
`myWord: .word 255`	16	2
`myLong: .long 2703`	32	4
`myByte: .byte '#'`	8	1

Este o declarare în secţiunea `.data` de forma `x: y: .long 15` validă? Argumentaţi răspunsul. RAH:

Este validă: în prima locaţie liberă din memorie se declară x. Nu se ştie ce tip de date sau ce valoare are x, se ştie doar că x începe la adresa respectivă. Se va repeta procesul pentru y: se caută prima adresă liberă din memorie (aceea este însăşi adresa lui x) şi se vor aloca 4B pentru valoarea 15. Prin urmare, x şi y există la aceeaşi locaţie în memorie

Scrieţi un apel de sistem WRITE (codul apelului este 4) pentru şirul de caractere `str2`, astfel încât să afişaţi, la `standard output`, întreg conţinutul şirului de caractere şi nimic altceva în plus.

```
movl $4, %eax        movl $16, %edx
movl $1, %ebx        int $0x80
movl $str2, %ecx
```

Exerciţiul 2 (0.5p) *Completaţi următorul tabel, cu valorile corecte în bazele 2, 8 şi 16.*

Baza 2	Baza 8	Baza 16
0100 0001 0010 0000 0000 0000 0000 0000	1 01100000000	41 20 00 00

Exerciţiul 3 (1p) *Fie următorul program, scris în* `assembly` *(procesor Intel x86, sintaxa ATET):*

```
.data
    x: .long 0x41200000
.text
.global main
main:
    movl $8, %eax
    mull x
et_exit:
    movl $1, %eax
    xorl %ebx, %ebx
    int $0x80
```

În hexa: x ← 0x41200000
eax ← 0x8
Înmulţirea (în hexa):
41200000 · 8 = 209000000
2·8(10) = 16(10) = 10(16)
4·8(10) = 32(10) = 20(16)

Executăm, în `debugger`*, următoarele comenzi:*

```
b main; b et_exit; run; c; i r edx
```

Valoarea pe care o vom obţine în registrul EDX este 2

2 | 09000000
%edx %eax

Exerciţiul 4 (1p) *Considerăm că, în programul de la exerciţiul anterior, după* `mull x` *inserăm instrucţiunile*

```
movl $2, %ebx
div %ebx
```

(restul programului rămâne identic, deci după aceste două instrucţiuni urmează `et_exit`*)*

Executăm, în `debugger`, următoarele comenzi:

```
b et_exit; run
```

Împărţirea (în hexa):

209000000 | 2
104800000

10(16) = 16(10)
(10 : 2)(16) = (16 : 2)(10) = 8(10)

2 = 0
0 = 9
8
10
10 = 8
8 = 0
....

Scanné avec CamScanner

1 | 04800000
%edx %eax

Descrieți ce ar trebui să se întâmple în acest punct, utilizând modul în care se execută instrucțiunea **div**:

Ar trebui să se efectueze împărțirea, al cărei cât să fie pus în %eax și rest în %edx. Însă câtul este mai mare de 32 de biți, conform calculelor anterioare. Instrucțiunea div nu poate pune câtul în %edx în aceste condiții, prin urmare programul se încheie cu o excepție.

Ce se întâmplă dacă modificăm atribuirea în EBX să fie **movl $16, %ebx**? (înainte de a executa **div %ebx**)

Împărțirea la 16 în zecimal este echivalentă cu împărțirea la 10 în hexa. După div, în %eax se va afla valoarea 0x20900000, iar în %edx valoarea 0. Împărțirea s-a efectuat cu succes, întrucât rezultatele au dimensiunile potrivite.

209000000 : 10 = 20900000

Exercițiul 5 (1p) *Fie următorul program, scris în* ***assembly*** *(procesor* ***Intel x86****, sintaxa* ***AT&T****):*

```
.data
    x: .long 18
    y: .long 20
.text
.global main
main:
    movl x, %eax ⇒ eax ← 18
    addl y, %eax ⇒ eax ← 38
    subl $2, %eax ⇒ eax ← 36
    xorl %edx, %edx ⇒ %edx ← 0
    divl y ⇒ eax ← 1, edx ← 16
    addl %edx, %eax ⇒ eax ← 17
    mul %eax ⇒ eax ← 289, edx ← 0
    sub %edx, %eax ⇒ eax ← 289
    decl %eax ⇒ eax ← 288
    cmp $272, %eax
    je et2
et1:
    movl $1, %ebx ⇒ ebx ← 1
    jmp et_exit
et2:
    movl $2, %ebx
et_exit:
    mov $1, %eax
    xorl %ebx, %ebx
    int $0x80
```

Completați valorile regiștrilor în momentul în care executarea programului ajunge în dreptul etichetei **et_exit** *(după ce am rulat în* ***debugger*** *comenzile* **b et_exit; run; i r**):

EAX	EBX	EDX
288	1	0

Exercițiul 6 (1p) *Fie următorul program, dezvoltat în limbajul de asamblare al procesorului* ***Intel x86****, sintaxa* ***AT&T****:*

```
.data
.text
.global main
main:
    movl $0xa5cf05c0, %eax   (eax ← 0xa5cf05c0)
    xorl %ecx, %ecx ecx ← 0
    xorl %ebx, %ebx ebx ← 0
    movw %ax, %cx            (cx ← 0x05c0)
    movb %ch, %cl ⇒ cx ← 0x0505
    xorb %ch, %ch ⇒ cx ← 0x0005 = 0x5
    movb %ah, %bl ⇒ bx ← 0x05 = 0x5
    movb %al, %bh ⇒ bx ← 0xc005
et_while:
    addb %bl, %bh
    loop et_while
et_exit:
    movl $1, %eax
    xorl %ebx, %ebx
    int $0x80
```

ecx ← 5: bx: 0xc505
ecx ← 4: bx: 0xca05
ecx ← 3: bx: 0xcf05
ecx ← 2: bx: 0xd405
ecx ← 1: bx: 0xd905

loop decrementează ecx automat înainte să verifice dacă e egal cu 0. *

Valoarea stocată în dreptul etichetei **et_exit** *în registrul EBX (în baza 16) este:* 0xd905

Exercițiul 7 (1p) *Explicați dacă următoarele două secvențe de cod accesează elementul de pe indexul al doilea al unui array de întregi, stocat în RAM la adresa cu numele v. Sunt secvențele de cod echivalente? Dacă da, de ce? Dacă nu, ce accesează fiecare, în parte?*

metoda de adresare a memoriei în x86: a(b,c,d) = b + c·d + q

a. **mov $v, %edi; movl $2, %ecx; movl (%edi, %ecx, 4), %ebx**

b. **mov $v, %edi; addl $8, %edi; movl $0, %ecx; movl (%edi, %ecx, 4), %ebx**

a. edi ← $v, ecx ← 2, ebx ← ecx · 4 + edi = 2 · 4 + edi = $v + 8
b. edi ← $v, edi ← edi + 8 = $v + 8, ecx ← 0, ebx ← ecx · 4 + edi = 0 + edi = $v + 8
Prin urmare, a și b sunt echivalente.

Exercițiul 8 (2p) *Scrieți o secvență de cod în* **main** *care să calculeze suma pătratelor elementelor dintr-un array. Considerați că lungimea array-ului este* **n**, *iar numele lui este* **v**. *Aceste informații sunt deja în secțiunea* **.data**. *Soluția voastră trebuie să înceapă de la eticheta* **main**.

* Adică: se intră în et_while, se execută instrucțiunile înainte de loop, apoi, când se ajunge la loop, se decrementează întâi %ecx, apoi se verifică și condiția de final: dacă e 0.

Scanné avec CamScanner

```
main:
    movl $v, %edi      ← mutăm pe v, %edi
    xorl %ecx, %ecx    ← indicele elementului curent în tablou
    xorl %ebx, %ebx    ← suma pătratelor elementelor
et_loop:
    cmpl %ecx, n       ← dacă am ajuns la finalul tabloului
    je et_exit         ← se încheie programul
    movl (%edi, %ecx, 4), %eax ← altfel, punem elementul curent din tablou în %eax
    mul %eax           ← îl ridicăm la pătrat
    incl %ecx          ← trecem la următorul element
    addl %eax, %ebx    ← adăugăm pătratul la sumă
    jmp et_loop        ← repetăm instrucțiunile

et_exit:
    movl $1, %eax
    xorl %ebx, %ebx
    int $0x80
```

Scanné avec CamScanner